

SPRINT 4

Equipo 6 - FirEye

Proyecto de Robótica

Manuel Pérez García

Yixuan Chen

Pablo Alejandro Chasi Cajamarca

Imanol Figuera Parras

Greysy Burgos Salazar

Yilun Jiang

Fecha de creación: 02/06/2026

CONTENIDO

Contenido	2
Tabla de ilustraciones	3
1. Fase de ideación y selección	4
1.2 ¿Por qué elegimos "FirEye"?	4
2. Análisis de empatía y contexto	5
2.2 Perfil del Usuario (UX)	5
2.3 Propósito (Corazón)	5
3. Definición e identidad del producto	6
3.2 Propuesta de Valor (Cerebro)	6
3.3 Modelo de Negocio	6
3.4 Identidad de Marca y Personalidad	6
4. Referencias y lecturas (Fase de vigilancia)	7
4.2 Robótica de última generación: aplicaciones en la vida diaria (HP Tech Takes)	7
4.3 TrackReitar – Lucha contra el fuego (Leotronics)	8
4.4 Robots contra incendios: tecnología e innovación (Cottés Group)	9
5. MOCKUPS	10
5.1. MoodBoard	11
5.2. LowFis	12
5.3. HighFis	15
6. Funcionalidades	20
9. Arquitectura del Sistema	29
10. Explicación de la base de datos (planteada)	30
11. ESTRUCTURA DE REPOSITORIOS Y ORGANIZACIÓN DEL PROYECTO	32
12. PILA TECNOLÓGICA Y HERRAMIENTAS UTILIZADAS	33
13. Reflexión crítica sobre el trabajo en grupo	34
14. Conclusión	34
Tabla historial de versiones	35

TABLA DE ILUSTRACIONES.

Ilustración 1 Logo base Fireye	7
Ilustración 2 Logo principal Fireye.....	10
Ilustración 3 Moodboard Fireye	11
Ilustración 4 LowFi del DashBoard.....	12
Ilustración 5 LowFi del Menú	12
Ilustración 6 LowFi de las notificaciones	13
Ilustración 7 LowFi de la configuración.....	13
Ilustración 8 LowFi de las notificaciones	14
Ilustración 9 HighFi de la Landing Page	15
Ilustración 10 HighFi del Dashboard	16
Ilustración 11 HighFi del Inicio de Sesión	16
Ilustración 12 HighFi del Menú	17
Ilustración 13 HighFi del botón del pánico	17
Ilustración 14 HighFi de las notificaciones	18
Ilustración 15 HighFi de la configuración	19
Ilustración 16 HighFi de la configuración II	19
Ilustración 17 HighFi de la configuración III	20
Ilustración 18 Esquema nodos lanzamiento entorno	20
Ilustración 19 Esquema nodos Nav Automatica	21
Ilustración 20 Testeo código python.....	23
Ilustración 21 Testeo Bags	23
Ilustración 22 Esquema de nodos general	24
Ilustración 23 Demostración procesado de imagen	25
Ilustración 24 Arquitectura del sistema	29
Ilustración 25 Diagrama de la base de datos.....	31

1. FASE DE IDEACIÓN Y SELECCIÓN

Durante la etapa de concepción, el equipo evaluó diversas ideas problemáticas y sociales. Se plantearon tres propuestas principales para el uso del robot (turtlebot3):

1. Asistente para personas de la tercera edad: un robot enfocado en el acompañamiento doméstico, recordatorio de medicación y monitorización de salud básica.
2. Gestor doméstico de reciclaje: un sistema robótico para la clasificación automática de residuos en el hogar, facilitando la tarea de reciclar al usuario.
3. Robot vigilante de incendios (Idea ganadora): un sistema de patrullaje autónomo para entornos rurales y fincas privadas. Aquí nace FirEye.

1.2 ¿Por qué elegimos "FirEye"?

La decisión elegir el robot vigilante de incendios se basó en los siguientes puntos clave detectados en BetaCards:

- Impacto Ambiental: Ataca un problema crítico en España sobre todo en la época de verano: los incendios forestales y rurales.
- Viabilidad Técnica: El uso del Turtlebot3 permite un desarrollo ágil en navegación autónoma y detección mediante visión artificial.
- Necesidad real: Existe un nicho de mercado claro en propietarios de fincas rurales (como nuestro usuario, *Antonio Delgado*) que sienten inseguridad ante incendios fortuitos o provocados.

FirEye se define como el primer sistema de vigilancia rural que patrulla y protege el pulmón de nuestros pueblos. Para el prototipo inicial nos centramos en la navegación y detección de incendios, sin embargo nos gustaría que fuera 100% autosuficiente mediante la integración de placas solares, para cubrir la necesidad de operar en fincas remotas donde no existen puntos de carga.

2. ANÁLISIS DE EMPATÍA Y CONTEXTO.

Definición del Desafío:

"Crear el primer sistema de vigilancia rural que patrulla y protege el pulmón de nuestros pueblos, detectando riesgos invisibles para evitar desastres ambientales."

High Concept:

"El primer sistema de vigilancia rural que patrulla y protege el pulmón de nuestros pueblos, detectando riesgos invisibles para evitar desastres ambientales."

2.2 Perfil del Usuario (UX).

Para diseñar FireEye, nos hemos centrado en las necesidades de un usuario real que representa el corazón de la España rural:

- **Nombre:** Antonio Delgado.
- **Edad:** 60 años.
- **Localización:** Burgohondo, Ávila (Castilla y León).
- **Ocupación:** Carnicero a punto de jubilarse. Ha invertido sus ahorros en una parcela donde sueña vivir su retiro.
- **Sus motivaciones:** Cuidar su patrimonio, mantener la vida en el campo y confiar en que su hogar está seguro. Le importan la familia y la salud.
- **Sus frustraciones:** Siente una profunda inseguridad y miedo por el peligro de los incendios constantes en los alrededores de su finca. Siente que, si ocurre algo, no tiene herramientas para reaccionar a tiempo.

2.3 Propósito (Corazón).

Siguiendo con la metodología de Simon Sinek, hemos definido el *alma* de nuestro proyecto para conectar emocionalmente.

¿Por qué? (Propósito): Trabajamos en erradicar el miedo de vivir a las afueras de los pueblos por los incendios desprevénidos y descontrolados. Queremos garantizar la salud de nuestros ecosistemas y asegurar un futuro sostenible para el medio ambiente, permitiendo que la gente vuelva a sentirse segura en el campo.

¿Cómo? (Proceso): mediante una solución tecnológica que destaca por ser segura, confiable y rápida, actuamos donde el ojo humano no llega, de forma constante y autónoma.

¿Qué? (Resultado): Un robot vigilante inteligente (FirEye) capaz de patrullar terrenos, detectar anomalías y activar protocolos de emergencia inmediatos.

3. DEFINICIÓN E IDENTIDAD DEL PRODUCTO.

3.2 Propuesta de Valor (Cerebro).

FirEye no solo es tecnología, es una solución diseñada para aportar valor en tres niveles fundamentales para el cliente:

- Funcional: Reduce riesgos de incendios y costes de vigilancia en la fina. Además, ahorra tiempo al propietario al automatizar el patrullaje.
- Cambio de vida: Ofrece esperanza y autorrealización, permitiendo que las personas mantengan sus proyectos de vida en el entorno rural de forma sostenible.

3.3 Modelo de Negocio.

Para asegurar que FirEye sea un proyecto viable y escalable, hemos definido la siguiente estrategia: Dirigirnos directamente a consumistas (B2C), como propietarios de fincas, aunque con posibilidad de escalar a gobiernos locales.

Apostaremos por la venta directa, el usuario adquiere el robot y el sistema de control como un producto en propiedad.

Nos construimos como una Start-up tecnológica bajo una figura de empresa privada que busca equilibrar el beneficio económico con el impacto ambiental.

3.4 Identidad de Marca y Personalidad.

La marca FirEye no es solo un nombre, es una declaración de intenciones:

Fire + Eye: el ojo que vigila el fuego.

Su personalidad se basa en dos arquetipos:

1. Arquetipo principal: El Héroe, ejercer valentía y proteger a los demás. FirEye es el *salvador* que actúa ante la adversidad con competencia y coraje tecnológico.

2. Arquetipo secundario: El Cuidador, ayudar a los demás, refleja nuestra parte humana, el deseo de proteger el patrimonio de los usuarios y la salud de nuestros bosques.

Identidad visual: El logotipo ha sido diseñado bajo los principios de simplicidad y funcionalidad, asegurando que sea reconocible.

- Simbología: El logo fusiona de forma abstracta un ojo y una llama. El círculo central actúa como la pupila (la cámara del Turtlebot3/el "ojo" del vigilante) y, al mismo tiempo, el espacio negativo y las formas envolventes sugieren la silueta de una llama controlada.

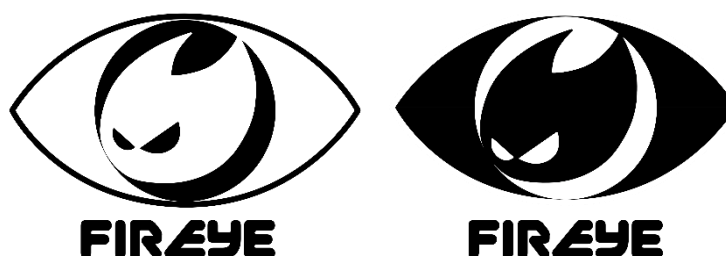


Ilustración 1 Logo base Fireeye

El logo se quedará en colores de blanco y negro. Pero la estética 100% de todo el proyecto aun no esta decidida del todo, pero rondara el minimalismo del logotipo.

4. REFERENCIAS Y LECTURAS (FASE DE VIGILANCIA).

Para garantizar que Fireeye no solo sea una idea creativa, sino una solución viable, técnica y comercialmente, hemos basado el desarrollo del Sprint 0 en un proceso de investigación activa.

4.2 Robótica de última generación: aplicaciones en la vida diaria (HP Tech Takes).

[Robótica de última generación: aplicaciones en la vida diaria < Tech Takes Blog - HP.com Colombia](#)

El artículo destaca la integración de la robótica y la inteligencia artificial en el día a día. Explica cómo los robots ya están presentes en hogares, salud, educación y logística. Señala que la combinación de IA y automatización permite mayor

autonomía, eficiencia y capacidad de toma de decisiones. El enfoque principal es que la robótica cotidiana es una tendencia creciente que transformará múltiples sectores.

IDEAS PRINCIPALES:

- Integración de inteligencia artificial con robótica.
- Robots cada vez más autónomos y conectados.
- Transformación de sectores como salud, educación y logística.

PROS:

- Mayor eficiencia y capacidad de toma de decisiones automática.
- Innovación constante y mejora de servicios.

CONTRAS:

- Alto coste de implementación.
- Necesidad de actualización y mantenimiento continuo.

4.3 TrackReitar – Lucha contra el fuego (Leotronics)

[TrackReitar Lucha contra el fuego - LeoTronics Robotics](#)

La página presenta un robot de orugas diseñado específicamente para combatir incendios en entornos extremos. Incluye sistemas de extinción de alta capacidad, cámaras térmicas y control remoto o semiautónomo. Está orientado a escenarios industriales y situaciones de alto riesgo donde la seguridad humana es prioritaria.

IDEAS PRINCIPALES:

- Robot de orugas diseñado para entornos extremos.
- Sistema de extinción de alta capacidad.
- Control remoto y opciones semiautónomas.

PROS:

- Alta resistencia y robustez industrial.
- Especialización en incendios de gran magnitud.

CONTRAS:

- Coste elevado de adquisición.

- Enfoque específico que limita su uso a escenarios concretos.

4.4 Robots contra incendios: tecnología e innovación (Cottés Group)

[Robots contra incendios: tecnología e innovación en la lucha contra el fuego y el humo | Cottés Group](#)

La página describe el uso de robots en la prevención y extinción de incendios. Explica que estas máquinas pueden acceder a zonas peligrosas, medir temperatura y gases, transmitir información en tiempo real y reducir el riesgo para los bomberos. También menciona el uso complementario de drones y sistemas de análisis de datos. Se resalta que el objetivo principal es mejorar la seguridad y eficiencia en emergencias.

IDEAS PRINCIPALES:

- Uso de robots para acceder a zonas de alto riesgo.
- Recopilación de datos en tiempo real (temperatura, gases, humo).
- Complemento al trabajo humano en emergencias.

PROS:

- Mayor seguridad para los bomberos.
- Mejor análisis de la situación antes de intervenir.

CONTRAS:

- Tecnología todavía en desarrollo.
- Dependencia de sistemas electrónicos en condiciones extremas.

El desarrollo de la inteligencia de Fireye no solo se nutre de la tecnología comercial actual, sino que se apoya en el ecosistema de innovación de la Universitat Politècnica de València. Como parte de nuestra vigilancia tecnológica, hemos identificado referentes académicos en el área de robótica y visión artificial dentro de la UPV, cuyas publicaciones y mentorías guiarán la implementación técnica del robot en los próximos Sprints. Esta colaboración nos permite integrar algoritmos avanzados de detección y navegación con la garantía de rigor académico.

5. MOCKUPS.

En este sprint hemos añadido también los mockups de cómo se verá nuestra web en los siguientes Sprints. Hemos hecho LowFis y Highfis, para tener en cuenta tanto como se verá a la hora de planificarlo y como se verá con todos los colores añadidos.

En cuanto al diseño, decidimos modificar el logo para darle una estética más colorida a FIREYE:



Ilustración 2 Logo principal Fireeye

5.1. MoodBoard

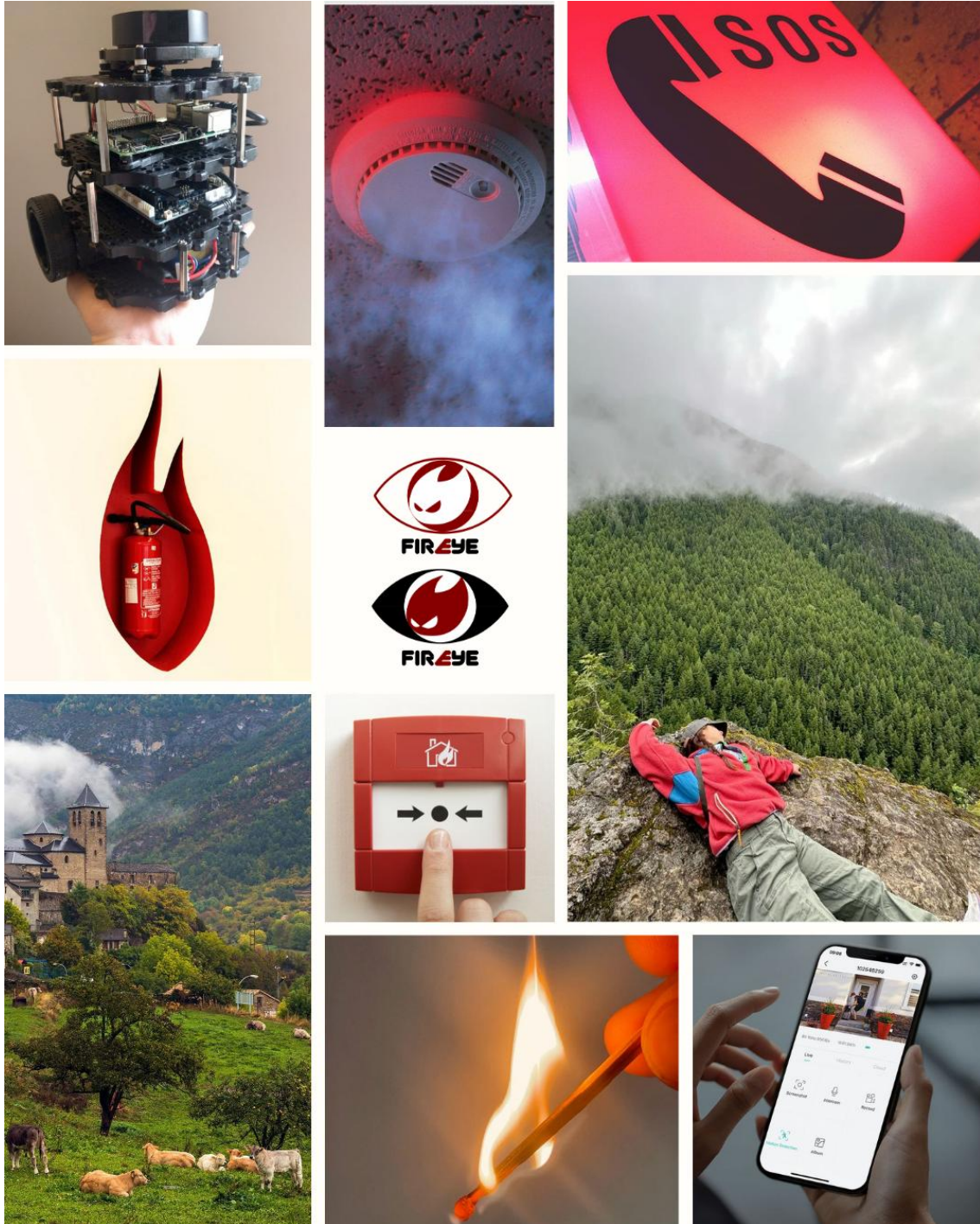


Ilustración 3 Moodboard Fireye

5.2. LowFis

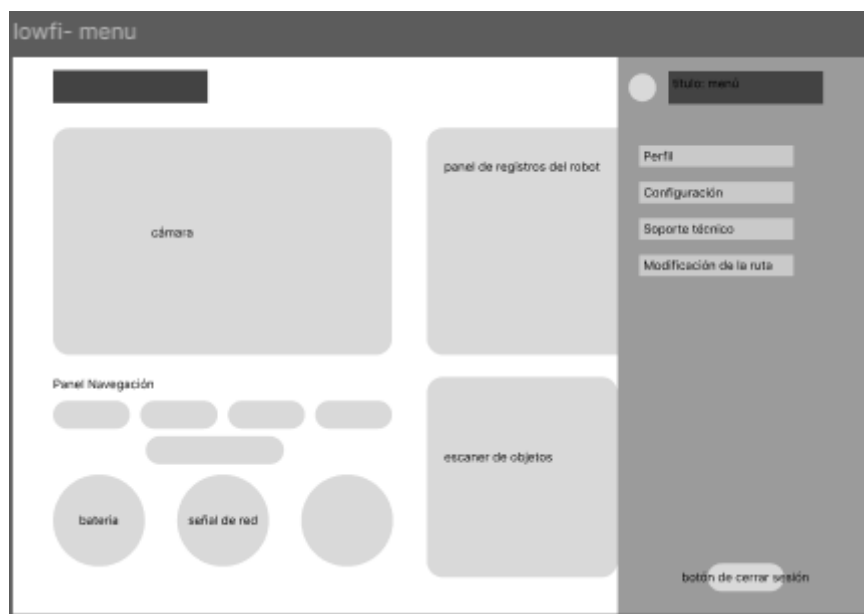


Ilustración 4 LowFi del Dashboard

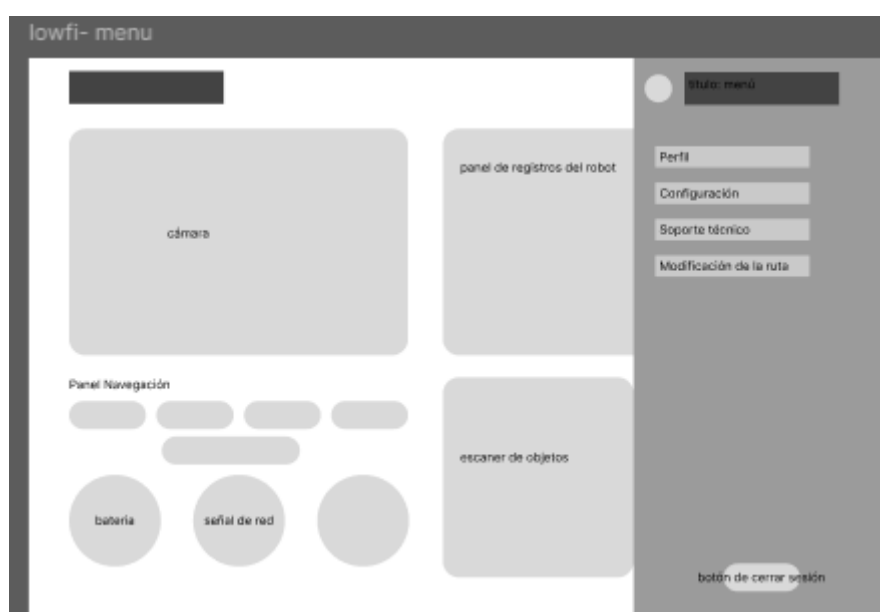


Ilustración 5 LowFi del Menú

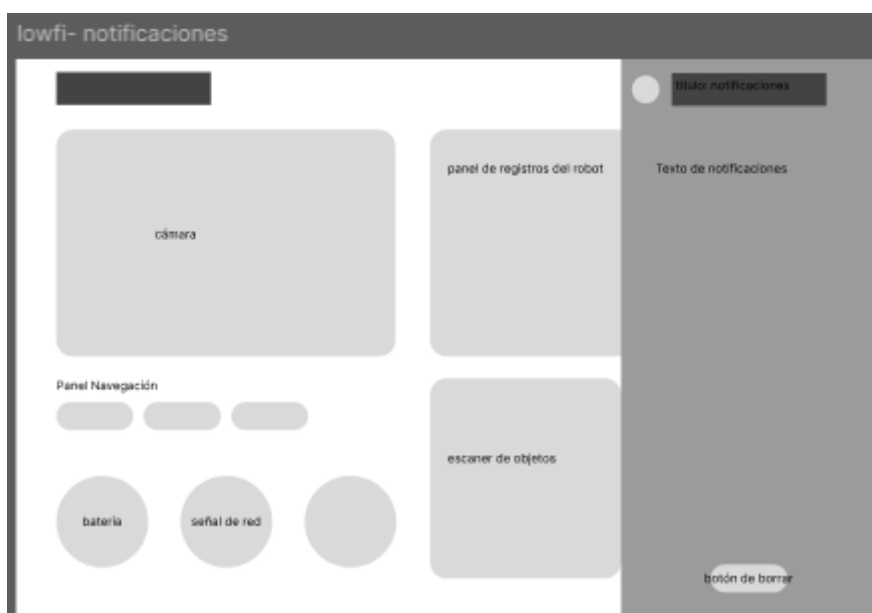


Ilustración 6 LowFi de las notificaciones



Ilustración 7 LowFi de la configuración

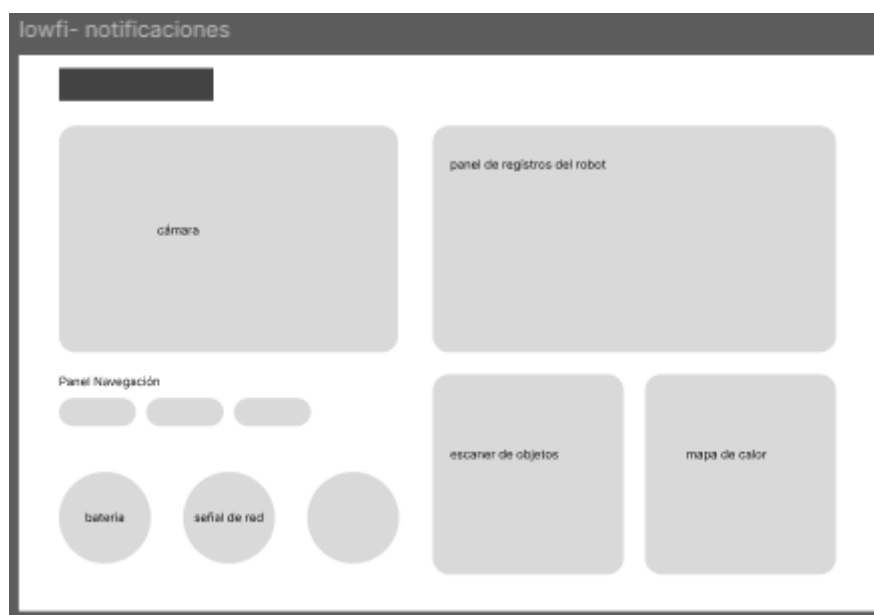


Ilustración 8 LowFi de las notificaciones

5.3. HighFis



FIREYE

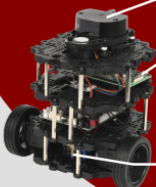
El primer sistema de vigilancia rural que patrulla y protege el pulmón de nuestros pueblos, detectando riesgos invisibles para evitar desastres ambientales.



PROTÉGETE CON NOSOTROS



HARDWARE DE PRECISIÓN



360° LiDAR
Para detectar obstáculos y anomalías térmicas

Raspberry Pi
Combina dos núcleos de procesamiento que trabajan en equipo, garantizando una vigilancia precisa y una respuesta inmediata ante el Fuego

Li-Po Battery
Hasta 24 horas de vigilancia continua

ADQUIERE FIREYE EN:

Puedes conseguir el kit completo de Vigilancia Rural a través de nuestros distribuidor oficial autorizado. Haz clic en tu plataforma de confianza para gestionar el envío:



Una vez completada la compra de tu unidad Fireye, recibirás en tu correo electrónico un código de activación único. Este proceso garantiza la Máxima Seguridad de tu propiedad:

1. Identificador Único (ID): Se te asignará una ID de Usuario vinculada exclusivamente al número de serie de tu robot.
2. Contraseña de Seguridad: Recibirás una clave de acceso encriptada para entrar en tu Dashboard personal.



¿Quién es Fireye?

Equipo Fireye

Preguntas frecuentes

© 2025 Fireye. Todos los derechos reservados.

Ilustración 9 HighFi de la Landing Page

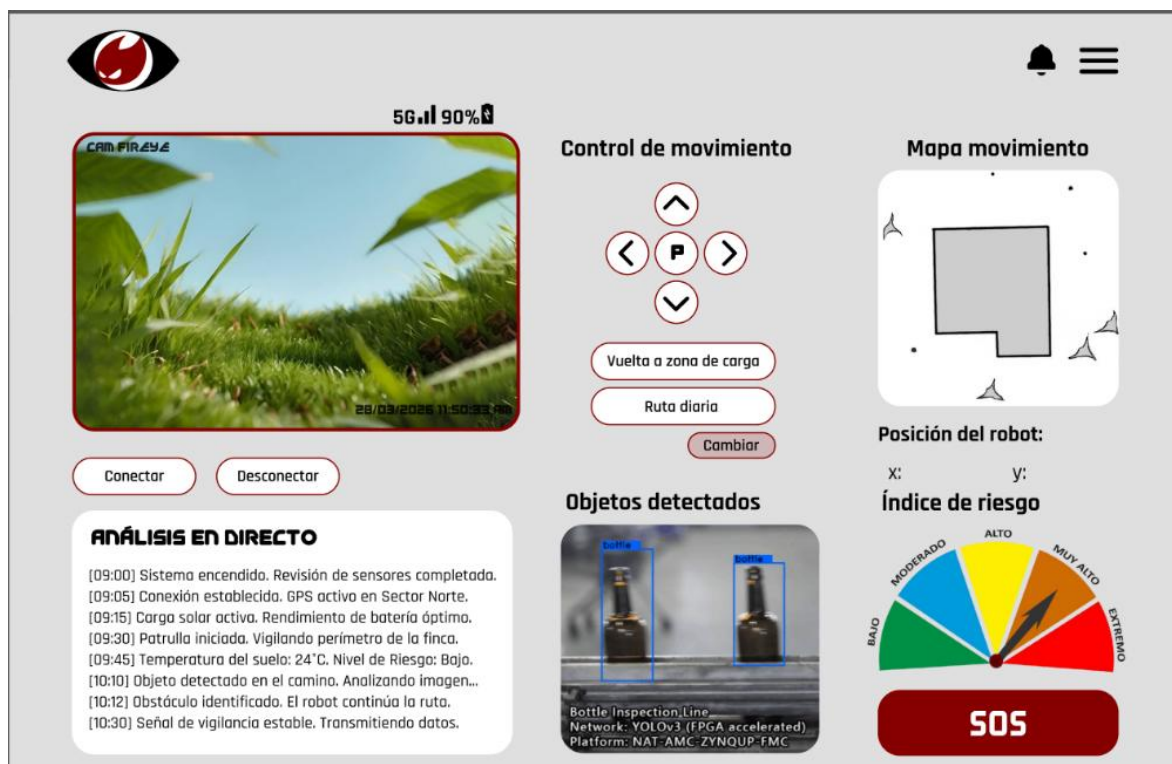


Ilustración 10 HighFi del Dashboard



Ilustración 11 HighFi del Inicio de Sesión



Ilustración 12 HighFi del Menú



Ilustración 13 HighFi del botón del pánico



Ilustración 14 HighFi de las notificaciones



Ilustración 15 HighFi de la configuración



Ilustración 16 HighFi de la configuración II

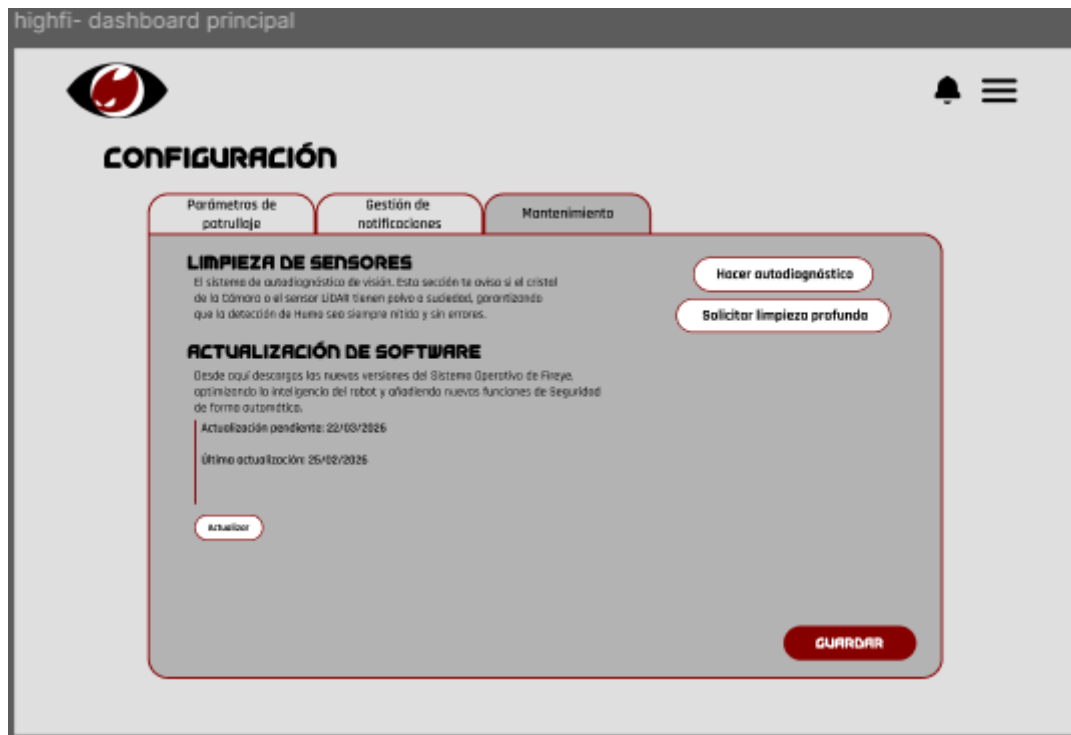


Ilustración 17 HighFi de la configuración III

6. FUNCIONALIDADES.

1. Lanzamiento del Entorno (Mundo y Robot):
 - El primer paso consiste en generar el entorno virtual donde operará el robot mediante Gazebo.
 - Simulación de Entorno: Se carga un mundo (archivo .launch.py) que define los obstáculos y paredes que el robot deberá detectar.
 - Carga del Modelo: Se inicializa el modelo del robot (en tu caso, el TurtleBot3 Burger), activando sus "músculos" (motores) y sus "sentidos" (sensor LiDAR).
 - Publicación de Transformadas (TF): El sistema comienza a calcular la posición del robot respecto al mundo (odom a base_link), permitiendo que ROS sepa dónde está cada parte del robot en todo momento.

Esquema diseño:

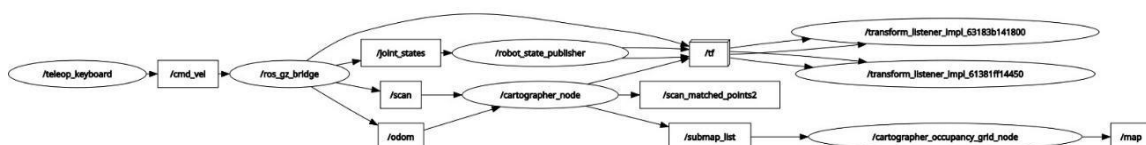


Ilustración 18 Esquema nodos lanzamiento entorno

Representación de la arquitectura del sistema de mapeo y localización simultáneos (SLAM). El flujo comienza en el teleop_keyboard, que traduce las pulsaciones de teclas en comandos de velocidad publicados en el topic

/cmd_vel. Estos comandos son recibidos por el nodo ros_gz_bridge, actuando como puente de comunicación. También observamos como tenemos los topics de:

- /scan: datos del sensor LiDAR para la detección de obstáculos y paredes.
- /odom: información de odometría que estima el desplazamiento del robot
- /joint_states: estado de las articulaciones, procesado por robot_state_publisher para mantener actualizada la estructura cinemática y las transformaciones de coordenadas /tf

2. Navegación Autónoma (Nav2):

Una vez el robot está en el mundo, el stack Nav2 se encarga de la inteligencia de movimiento:

- Mapas de Costos (Costmaps): El robot genera un mapa global para planificar la ruta larga y un mapa local para esquivar obstáculos imprevistos en tiempo real.
- Planificación y Control: Al recibir una meta (vía 2D Goal Pose), el sistema traza una línea de camino y envía comandos de velocidad (/cmd_vel) para seguirla de forma segura.
- Gestión del Ciclo de Vida: A través del Nav2 Dashboard, se controlan los estados de los nodos (Startup/Active), asegurando que el servidor de acciones esté listo para procesar objetivos.

Esquema diseño:

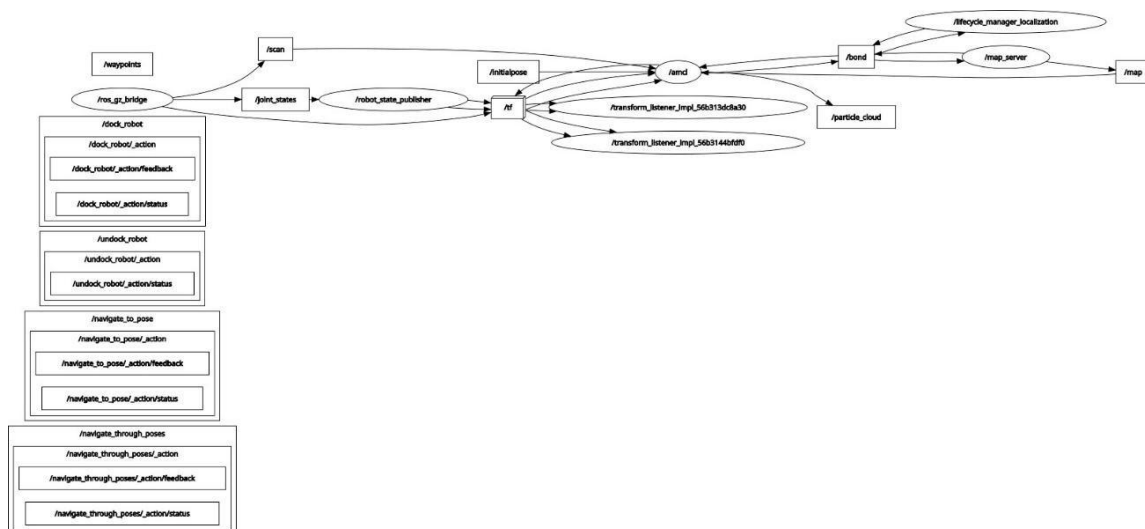


Ilustración 19 Esquema nodos Nav Automatica

Arquitectura de navegación autónoma basada en el ecosistema Nav2 y el algoritmo de localización AMCL. Aquí el robot ya dispone de un mapa estático y su objetivo es localizarse con precisión dentro de el para cumplir misiones de movimiento. El proceso comienza con el nodo `/map_server`, que carga y publica el mapa generado previamente en el topic `/map`, el funcionamiento es el siguiente: se comparan los datos en tiempo real del sensor laser `/scan` con el mapa estático, se recibe una `/initialpose` para una posición aproximada del robot para comenzar el proceso de localización y por ultimo `/particle_cloud`, que genera una nube de partículas que visualiza donde cree el sistema que se encuentra el robot hasta que la posición converge.

En el lateral izquierdo se observa la integración de los Action Servers de Nav2. Estos nodos gestionan tareas complejas mediante una estructura de objetivo-retroalimentación-resultado con `/navigate_to_pose`, recibe una meta específica y coordina al planificador y al controlador para llegar al destino. Con

`/navigate_through_poses`, permite definir rutas complejas que pasan por múltiples puntos de control y por ultimo `/dock_robot` y `/undock_robot` que gestionan los comportamientos específicos para maniobras de precisión, como el acoplamiento en estaciones de carga.

Un componente importante de este esquema es

`/lifecycled_manager_localization`, lo que significa que los nodos no solo se ejecutan, sino que pasan por estados, de esta forma el gestor se asegura que el sistema de localización este totalmente listo antes de permitir que el robot intente moverse, garantizando la seguridad operativa.

Finalmente, el topic `/tf`, centraliza las relaciones especiales entre el robot (`base_link`) y sus sensores. El nodo `/robot_state_publisher` se encarga de que todas las piezas del robot sepan dónde están respecto a las demás en tiempo real, permitiendo que la navegación sea precisa.

3. Registro y Validación (Bags y Estímulos)

Una funcionalidad crítica para el desarrollo profesional es la capacidad de capturar la actividad del sistema:

- Grabación de Estímulos: Se utiliza el comando `ros2 bag record` para guardar los mensajes publicados en tópicos como `/scan`, `/odom` y `/cmd_vel`.

- Reproducción de Trayectorias: Los archivos generados permiten reproducir los mensajes grabados, haciendo que el robot recorra la misma trayectoria inicial para testear el comportamiento del sistema ante estímulos idénticos sin necesidad de repetir la misión manualmente.
- Análisis de Datos: El archivo metadata.yaml proporciona información sobre el tiempo de grabación, los tópicos monitoreados y el tipo de mensajes, facilitando la auditoría del rendimiento del robot.

Test código Python:

```

grey@grey-Presti... x grey@grey-Presti... x grey@grey-Presti... x grey@grey-Presti... x
./test/test_initial_pose_pub.py:22:1: W293 blank line contains whitespace
./test/test_initial_pose_pub.py:29:1: W293 blank line contains whitespace
./test/test_initial_pose_pub.py:31:39: W291 trailing whitespace
./test/test_initial_pose_pub.py:32:28: W291 trailing whitespace
./test/test_initial_pose_pub.py:33:28: W291 trailing whitespace
./test/test_initial_pose_pub.py:38:39: Q000 Double quotes found but single
quotes preferred
./test/test_initial_pose_pub.py:39:1: W293 blank line contains whitespace
./test/test_initial_pose_pub.py:49:43: Q000 Double quotes found but single
quotes preferred
./test/test_initial_pose_pub.py:50:1: W293 blank line contains whitespace
./test/test_initial_pose_pub.py:53:48: Q000 Double quotes found but single
quotes preferred
./test/test_initial_pose_pub.py:55:33: W292 no newline at end of file
assert 1 == 0
>>>
proy_fireye_slam.test.test_pep257 test_pep257
<<< failure message
AssertionError: Found code style errors / warnings
assert 1 == 0
>>>

summary: 5 tests, 0 errors, 2 failures, 1 skipped
grey@grey-Prestige-16-AI-Studio-B1VFG:~/turtlebot3_ws$

```

Ilustración 20 Testeo código python

Test bags:

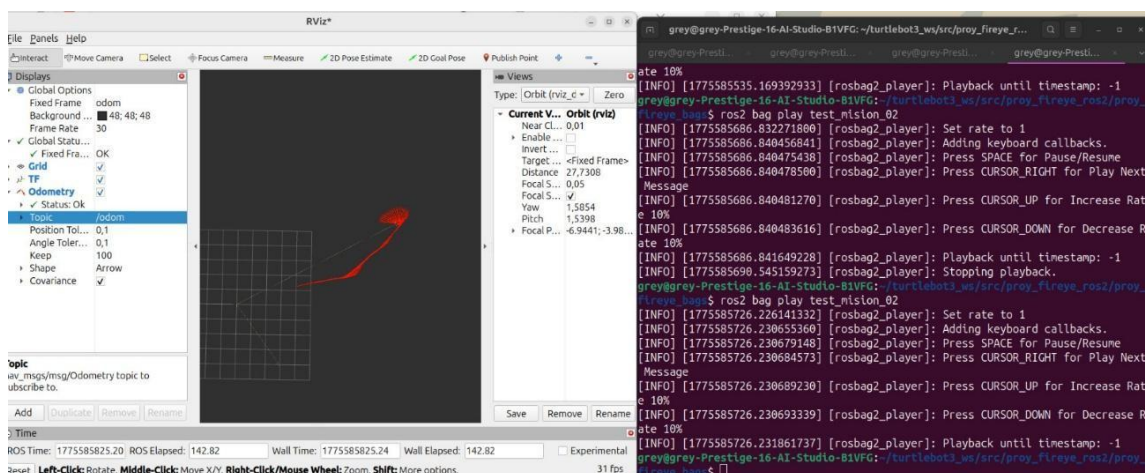


Ilustración 21 Testeo Bags

4. Interfaz de control y monitoreo web (Fireye RosWeb)

La interfaz actúa como el centro de mando remoto del robot, permitiendo al usuario final interactuar con el sistema ROS2.

- **Arquitectura de Comunicación:** Se utiliza el paquete `rosbridge_suite` para establecer un puente de comunicación vía WebSockets. Esto permite que la web, desarrollada en HTML5, JS y PHP, envíe y reciba mensajes JSON que se traducen a tópicos de ROS 2.
- **La web se suscribe a los tópicos de Sistemas Críticos,** visualizando dinámicamente el porcentaje de batería (`/battery_state`), la calidad de red y la velocidad lineal/angular del robot (`/odom`).
- **Control de Navegación Visual:** Mediante la integración con el stack Nav2, el usuario puede monitorizar el estado de las misiones y ver el "Análisis en Directo" de los registros del robot.
- **Protocolos de Emergencia (Botón del Pánico):** Tal como se muestra en los diseños de alta fidelidad (Imagen 10), la web permite la activación manual de alarmas y protocolos de aviso a emergencias en caso de detectar un foco de incendio confirmado.

Esquema de la arquitectura resultante:

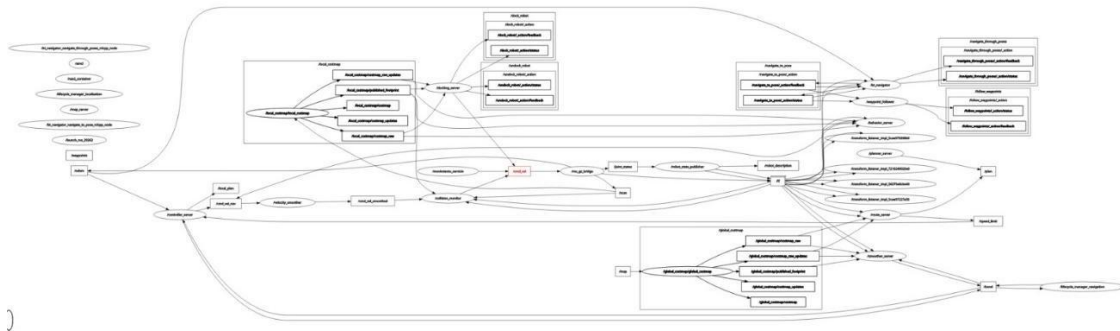


Ilustración 22 Esquema de nodos general

5. Sistema de visión artificial y reconocimiento de objetos.

Para dotar el robot de la capacidad de percibir e identificar elementos en su entorno hemos desarrollado un pipeline de visión artificial estructurado en 4 fases, integradas dentro del ecosistema de ROS2:

- **Adquisición y preprocesamiento de imagen:** Se implementó un nodo en ROS 2 encargado de suscribirse al flujo de video entrante desde la cámara del robot. Las imágenes capturadas son sometidas a un filtrado inicial que incluye la conversión a escala de grises y la aplicación de desenfoque gaussiano (GaussianBlur). Posteriormente, se realiza una transformación al espacio de color HSV, un paso fundamental para mitigar el ruido visual y

acondicionar la imagen para una detección cromática y de objetos más robusta.

- Segmentación y detección de características: A partir de las imágenes preprocesadas, el sistema aplica el algoritmo Canny para la detección de bordes estructurales y utiliza la función `findContours` para extraer la morfología de los elementos presentes en la escena. De forma paralela, se aplican máscaras de filtrado en el espacio HSV mediante la función `inRange`, aislando regiones que presentan propiedades de superficies metálicas o brillantes. A los candidatos resultantes de esta etapa se les superponen cajas delimitadoras preliminares.
- Análisis geométrico y clasificación: Una vez aislados los contornos, el sistema procede a analizar la topología de los mismos. Mediante algoritmos de aproximación poligonal (*approxPolyDP*), el cálculo del número de vértices y la evaluación de la relación de aspecto (*aspect ratio*), el nodo procesa las características morfológicas para clasificar de manera autónoma los objetos detectados en categorías específicas: Lata, Botella o Aerosol.
- Localización visual y anotación en el tiempo real: Como culminación del proceso de percepción, el sistema determina la ubicación exacta del objeto en el plano de la imagen. El algoritmo calcula el centroide de cada contorno válido utilizando momentos de imagen (`cv2.moments()`), obteniendo el punto central del objeto. Para proporcionar feedback visual en tiempo real, el sistema renderiza directamente sobre la transmisión de la cámara:
 - Un marcador central (punto rojo) indicando el centroide exacto.
 - Un bounding box (caja delimitadora verde) envolviendo el área del objeto.
 - Una etiqueta superpuesta que muestra la clasificación del objeto y sus coordenadas bidimensionales en la pantalla (ej. Lata 1 (511, 174)).

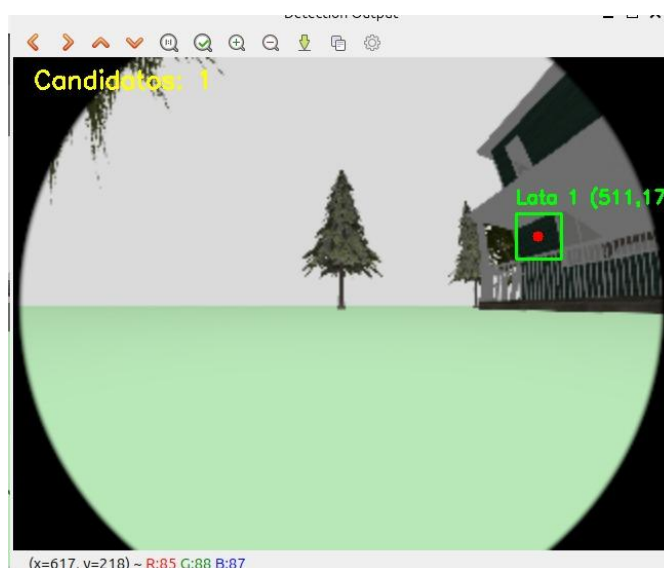


Ilustración 23 Demostración procesado de imagen

6. Representación cartográfica y localización en la interfaz web

Como extensión del panel de control y monitoreo web, se ha implementado un módulo de renderizado bidimensional interactivo que permite visualizar el mapa del entorno y rastrear la posición exacta del robot en tiempo real directamente desde el navegador. Esta funcionalidad se divide en los siguientes procesos:

- **Renderizado Dinámico del Mapa de Ocupación:** A través de la conexión por WebSockets (`rosbridge_suite`), el cliente web se suscribe directamente al tópico `/map`. Al recibir los mensajes de tipo `nav_msgs/msg/OccupancyGrid`, el sistema procesa la matriz de datos espaciales utilizando una función de renderizado personalizada (`draw_occupancy_grid.js`) sobre un elemento `<canvas>` nativo de HTML5. El algoritmo itera sobre la cuadrícula y evalúa la probabilidad de ocupación de cada celda, traduciéndola a un gradiente de color visual: blanco para espacios libres (valor 0), negro para obstáculos (valor 100) y gris para áreas no mapeadas o desconocidas.
 - **Transformación de Coordenadas Espaciales:** Para proyectar de forma precisa al robot dentro de este entorno cartográfico, la interfaz se suscribe de manera simultánea al tópico `/odom` para recuperar los datos de odometría. Dado que las dimensiones en ROS 2 y en la web son distintas, el sistema aplica una transformación geométrica a las coordenadas espaciales: toma la resolución y el origen del mapa (extraídos de sus metadatos .yaml) y convierte la posición cartesiana del robot a coordenadas de píxeles bidimensionales (X, Y) dentro del canvas.
 - **Visualización y Seguimiento en Tiempo Real:** Una vez calculada la traslación exacta, el sistema renderiza un marcador dinámico (representado como un punto geométrico de color verde) superpuesto al mapa de ocupación. Este ecosistema web está programado para limpiar y redibujar (`draw`) tanto el mapa como el marcador continuamente en respuesta a las actualizaciones de los tópicos, garantizando que el usuario obtenga una telemetría visual fluida e instantánea sobre la ubicación y las trayectorias del robot.
7. **Detección Avanzada de Objetos: Entrenamiento e Integración del Modelo YOLO.**
- Para robustecer la capacidad de percepción descrita en el apartado anterior, se ha implementado un sistema avanzado de reconocimiento visual basado en Deep Learning
- **Creación del Dataset y Entrenamiento.**
Se creó un dataset fotográfico propio con los objetos objetivo (latas, botellas y aerosoles). Las imágenes fueron etiquetadas manualmente mediante CVAT en formato YOLO 1.1, generando la siguiente estructura exportada (`obj.data`, `obj.names`, `train.txt`, `obj_train_data/`).

Con este dataset, se entrenó el modelo YOLO. Los resultados y pesos resultantes se almacenan en el repositorio (dataset/runs/detect/train/weights/), destacando:

- best.pt: Modelo entrenado con los mejores resultados.
 - last.pt: Último modelo guardado durante el entrenamiento.
 - best.onnx: Versión exportada en formato ONNX para facilitar integraciones multiplataforma.
- Paquete ROS2 de Visión.
- Se desarrolló un paquete específico en ROS2 para la integración del modelo, conteniendo la configuración estándar (package.xml, setup.py, setup.cfg, y directorios resource/ y test/).
- El núcleo funcional es el nodo **yolo_detector.py**, cuyo ciclo de ejecución es el siguiente:
1. Suscripción al topic de la cámara del robot.
 2. Conversión del mensaje de imagen de ROS a formato OpenCV mediante cv_bridge.
 3. Ejecución de la inferencia del modelo YOLO sobre el frame actual.
 4. Extracción de detecciones y trazado de cajas delimitadoras sobre la imagen.
 5. Publicación de la imagen anotada y de la información estructurada de los objetos detectados.
- El flujo de información resultante opera de la siguiente manera:
- Cámara del robot → Nodo YOLO → Detección de objetos → Publicación de resultados en ROS2**
- Validación y pruebas.
- La integración incluye el script independiente test_yolo.py, diseñado para comprobar la correcta carga del modelo y su capacidad de detección en imágenes estáticas sin necesidad de levantar toda la arquitectura ROS2. Finalmente, el modelo fue validado directamente en el entorno simulado de Gazebo, demostrando un reconocimiento efectivo y en tiempo real de los elementos distribuidos por el escenario.

8. Despliegue y Control en Hardware Físico.

Como culminación de las pruebas en simulación, se ha llevado a cabo la transición del sistema al hardware físico, logrando la teleoperación y navegación autónoma del robot real en un entorno físico. Esta funcionalidad valida la portabilidad del código desarrollado en ROS 2.

- **Arquitectura de Comunicación Distribuida:** El sistema opera bajo un modelo de red distribuida donde el PC actúa como estación base (nodo de control principal) y se comunica con el microordenador del robot físico a través de una red local inalámbrica (Wi-Fi), utilizando el middleware DDS (Data Distribution Service) nativo de ROS 2.
- **Interfaz de Control (RViz2):** Se utiliza RViz2 desde la estación base para visualizar en tiempo real la telemetría del robot físico (datos del LiDAR real y odometría de los motores). A través de esta misma interfaz, el operador puede enviar comandos de navegación interactivos (mediante 2D Goal Pose).
- **Navegación por Puntos de Control (Waypoints):** Se ha implementado un archivo de lanzamiento específico (`follow_waypoint.launch.py`) que permite cargar y ejecutar rutinas de movimiento. Este script coordina el stack de Nav2 para que el robot real siga una trayectoria predefinida de múltiples puntos en el espacio secuencialmente.
- **Análisis de Rendimiento y Limitaciones de Red:** Durante las pruebas en hardware físico se ha detectado una limitación crítica relacionada con el ancho de banda y la estabilidad de la red. Dado que el envío de los comandos de velocidad (`/cmd_vel`) y la recepción de datos de los sensores exigen una alta frecuencia de publicación, cualquier congestión en la red local provoca pérdida de paquetes o picos de latencia. Esto se traduce en interrupciones intermitentes del movimiento, donde el sistema de control del robot detiene los motores por motivos de seguridad al dejar de recibir comandos actualizados de la estación base.

9. ARQUITECTURA DEL SISTEMA.

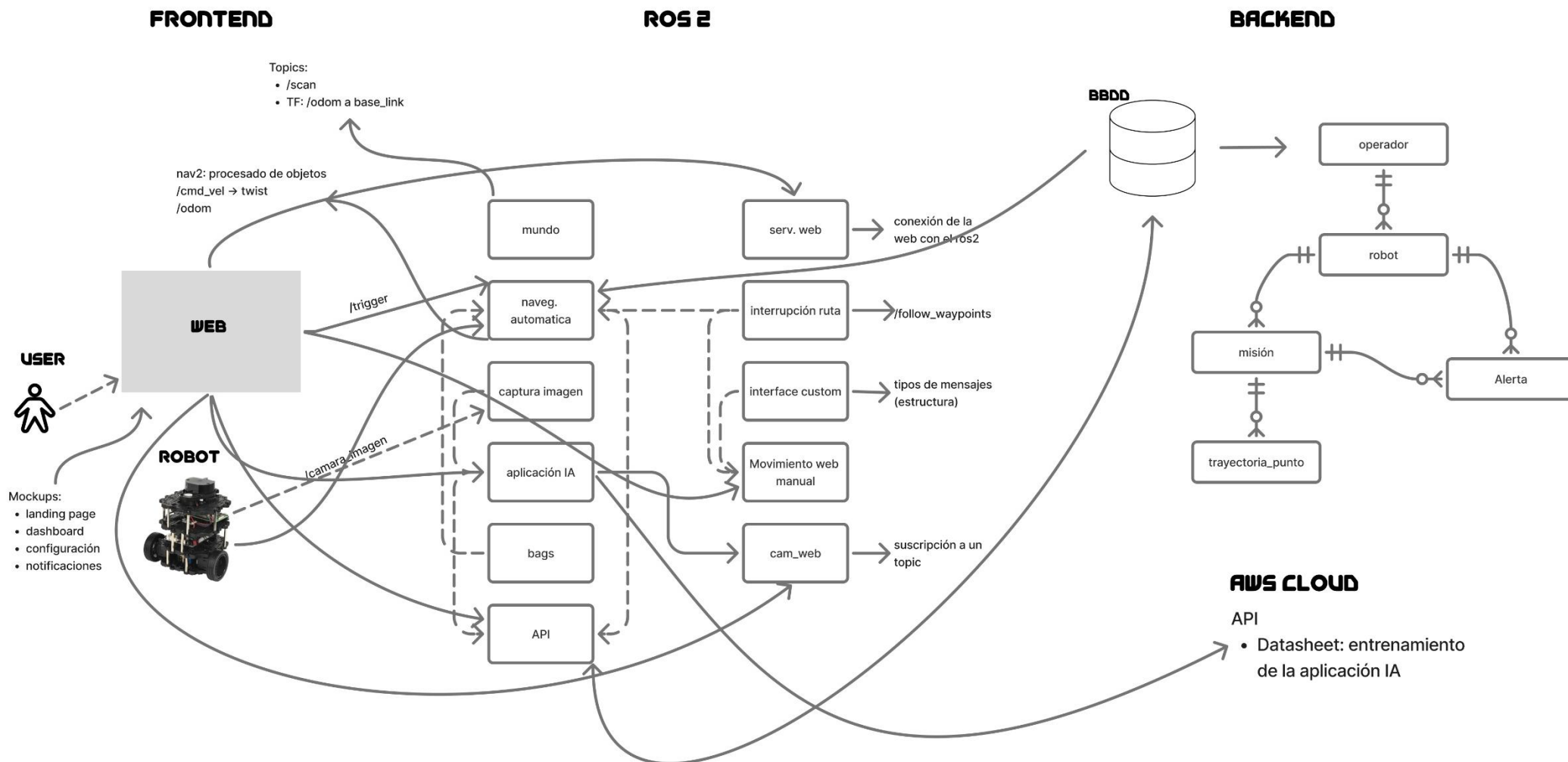


Ilustración 24 Arquitectura del sistema

10. EXPLICACIÓN DE LA BASE DE DATOS (PLANTEADA)

Para garantizar la trazabilidad, el registro histórico de misiones y la correcta gestión de la flota y sus operadores, el sistema cuenta con una base de datos relacional estructurada para almacenar la telemetría y los eventos críticos del robot. El diseño sigue un esquema de entidad-relación compuesto por las siguientes tablas principales.

- Entidad OPERADOR: Actúa como el nivel más alto de la jerarquía de usuarios. Almacena la información del personal encargado de monitorizar el sistema, incluyendo un identificador único (*id* como Primary Key), *nombre*, *email de contacto* y *el cargo que ocupa*.
- Entidad ROBOT: Representa el estado físico y lógico de cada unidad dentro de la flota. Guarda detalles como el modelo, el estado operativo en tiempo real (*activo*, *cargando*, *error*) y el *nivel_bateria*. Esta tabla se vincula directamente con el operador responsable a través de una clave foránea (*operador_id*), estableciendo que un operador puede supervisar múltiples robots.
- Entidad MISIÓN: Es el núcleo del registro de operaciones. Cada vez que el robot inicia una tarea de navegación o patrullaje, se genera un registro con un timestamp de inicio y fin, así como el resultado final de la misma (*completada*, *abortada*). Cada misión está estrictamente asignada a un robot específico mediante la clave foránea *robot_id*.
- Entidad TRAYECTORIA_PUNTO: Permite una reconstrucción granular del recorrido del robot. Contiene las coordenadas espaciales (*pos_x*, *pos_y*, *pos_z*) extraídas de la odometría en instantes de tiempo precisos (*capturado_en*). Una única misión contiene una multiplicidad de estos puntos métricos, vinculados mediante *misión_id*, lo que facilita la reproducción posterior de trayectorias.
- Entidad ALERTA: Fundamental para los protocolos de emergencia del sistema. Registra automáticamente cualquier anomalía o detección crítica generada por los sensores o el sistema de visión artificial. Incluye el tipo de alerta (*fuego*, *humo*, *batería_baja*), su ubicación exacta (*latitud*, *longitud*), el *nivel_confianza* de la detección (útil para el clasificador visual) y la *fecha_hora* del suceso. Las alertas mantienen una doble trazabilidad: saben qué robot las generó (*robot_id*) y durante qué misión ocurrieron (*misión_id*)

Flujo Relacional del Sistema:

La arquitectura garantiza la integridad referencial: un operador supervisa robots; un robot ejecuta misiones; una misión traza múltiples puntos de trayectoria; y de forma concurrente, tanto el robot como sus misiones generan registros de alerta que quedan vinculados permanentemente para futuras auditorías o análisis forense.

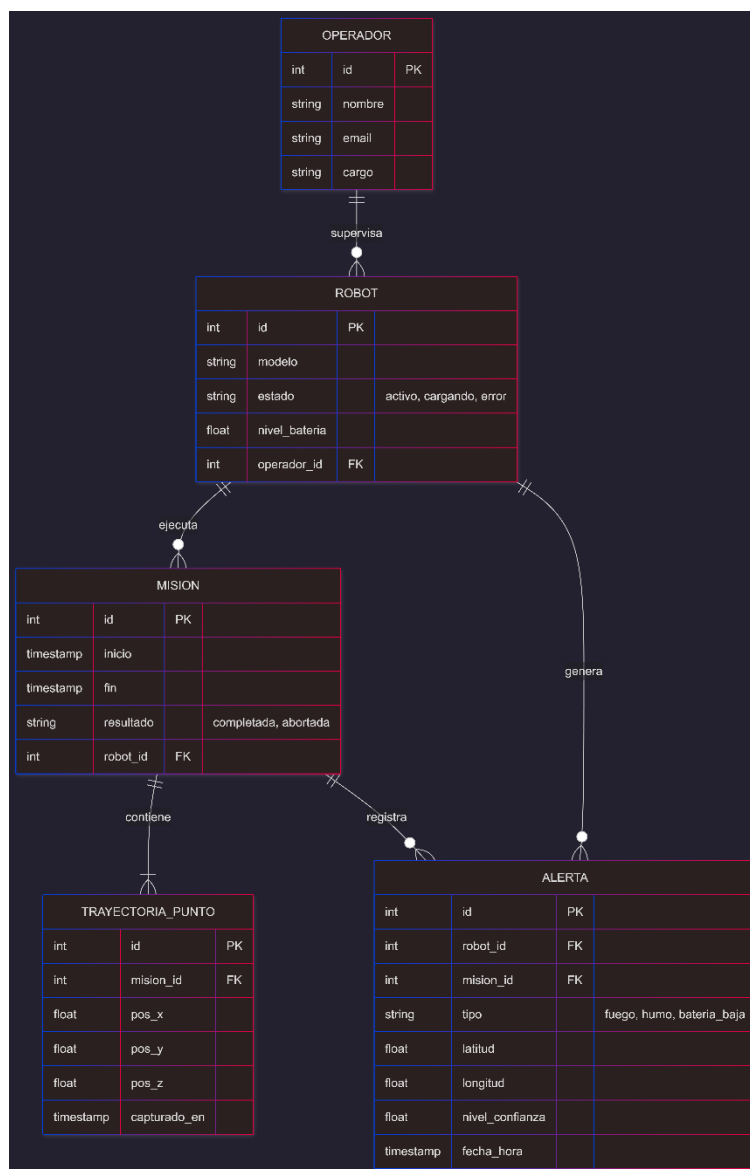


Ilustración 25 Diagrama de la base de datos

11. ESTRUCTURA DE REPOSITORIOS Y ORGANIZACIÓN DEL PROYECTO

El desarrollo del sistema se ha modularizado en dos repositorios de control de versiones independientes para separar la lógica de control robótico (ROS 2) de la interfaz de usuario (Web). Esta arquitectura garantiza un código mantenible, escalable y ordenado.

1. Repositorio ROS 2 (Lógica del Robot y Simulación) Este repositorio contiene el espacio de trabajo (*workspace*) principal del TurtleBot3 y concentra todos los paquetes encargados de la percepción, navegación y control físico o simulado del robot. Se debe alojar dentro de la carpeta *src/* del espacio de trabajo y se divide en los siguientes subpaquetes:

- **proy_fireye_ros2/**: Es el paquete principal que aloja el código central del robot. Aquí se encuentran los nodos dedicados a la visión artificial (OpenCV), el procesamiento de imágenes y la lógica base del comportamiento autónomo.
- **proy_fireye_mundo/**: Contiene los archivos de configuración y *launch files* necesarios para generar el entorno de simulación en Gazebo. Define las físicas, los modelos estáticos, obstáculos y paredes del escenario virtual.
- **proy_fireye_SLAM/**: Agrupa la configuración del ecosistema de navegación autónoma y mapeo. Incluye los parámetros de Nav2, mapas de costos (*costmaps*) y los algoritmos de localización (AMCL) que permiten al robot orientarse.
- **proy_fireye_bags/**: Directorio destinado a almacenar los registros de pruebas y simulaciones (archivos *.db3* y *metadata.yaml*). Permite la reproducción de trayectorias y el análisis de estímulos sin necesidad de repetir las misiones manualmente.

2. Repositorio RosWeb (Interfaz de Control y Monitoreo) Este repositorio conforma el *frontend* del sistema, funcionando como el centro de mando remoto del operador. Se comunica con el ecosistema ROS 2 a través de WebSockets utilizando *rosbridge_suite*. Su estructura es la siguiente:

- **index.html**: Archivo raíz y punto de entrada de la aplicación web. Contiene la estructura de la interfaz, los paneles de control y el *canvas* de renderizado del mapa.
- **src/**: Directorio de recursos (*assets*) y código fuente del cliente, subdividido en:
 - **css/**: Hojas de estilo que definen el diseño visual, la adaptabilidad (responsividad) y la estética del panel de mando.
 - **js/**: Archivos JavaScript que manejan la lógica de conexión (WebSockets).

- **img/ & fonts/ & html/**: Carpetas de soporte para recursos gráficos, tipografías personalizadas y fragmentos HTML modulares requeridos por la interfaz.

3. Documentación General

- **docs/**: Adicionalmente a la estructura de ejecución, el proyecto cuenta con un directorio transversal de documentación técnica. Este almacena los manuales de usuario, diagramas de arquitectura (como el modelo entidad-relación de la base de datos), especificaciones de diseño y guías de despliegue de ambos repositorios.

12. PILA TECNOLÓGICA Y HERRAMIENTAS UTILIZADAS.

El desarrollo de este sistema integra un conjunto de tecnologías modernas y estandarizadas en la industria robótica, divididas en dos grandes bloques: el ecosistema de control y simulación, y la arquitectura web para la monitorización remota.

1. Ecosistema de Robótica y Simulación (Backend)

Este bloque constituye el núcleo lógico y físico (simulado) del comportamiento del robot:

- **ROS 2 Jazzy Jalisco**: Actúa como el middleware y marco de trabajo principal que sustenta toda la arquitectura de software. Proporciona el sistema de comunicación basado en nodos, publicación/suscripción de tópicos y gestión del ciclo de vida necesario para la navegación y visión del robot.
- **Lenguajes C++ y Python**: El código fuente del robot se ha desarrollado utilizando un enfoque híbrido. Se emplea C++ para aquellos nodos que requieren alta eficiencia computacional y baja latencia (como el control a bajo nivel), mientras que Python se utiliza para la integración ágil de algoritmos complejos, procesamiento de datos y pipelines de visión artificial (OpenCV).
- **Gazebo Harmonic**: Empleado como motor de simulación de físicas. Permite modelar el comportamiento cinemático del TurtleBot3, la interacción con los sensores (LiDAR, cámara) y recrear con precisión el entorno tridimensional (obstáculos, paredes) antes de cualquier despliegue en hardware real.

2. Tecnologías de Interfaz de Usuario y Comunicación (Frontend)

Para la creación del centro de mando y la comunicación bidireccional con el ecosistema robótico, se utilizaron las siguientes tecnologías web:

- HTML5 / CSS3 / JavaScript: Conforman la tríada base para el desarrollo del frontend. HTML5 proporciona la estructura semántica y elementos interactivos (como el renderizado del mapa en un <canvas>), CSS3 asegura un diseño adaptable e intuitivo para el operador, y JavaScript gestiona toda la lógica de presentación y telemetría en tiempo real.
- rosbridge_suite: Funciona como la pieza clave de infraestructura de red. Este paquete levanta un servidor de WebSockets que actúa como puente, traduciendo los mensajes nativos de ROS 2 a formato JSON estandarizado, permitiendo que un entorno no-ROS (como un navegador web) pueda comunicarse con el robot.
- roslibjs: Es la biblioteca central del lado del cliente. Escrita en JavaScript, permite que la interfaz web establezca la conexión directa con el servidor, facilitando la suscripción a tópicos críticos (odometría, mapas, baterías) y la publicación de comandos directamente desde el navegador.

13. REFLEXIÓN CRÍTICA SOBRE EL TRABAJO EN GRUPO.

Haciendo balance sobre nuestro trabajo, la dinámica del grupo ha destacado por el compañerismo, la equidad y el compromiso de todos. Desde el primer momento intentamos organizarnos para que el reparto de tareas fuera lo más justo posible. Aunque, como ocurre siempre en estos trabajos, alguien termina tomando un poco más la iniciativa para conseguir cuadrarlo todo al final, sentimos que el esfuerzo general ha estado muy bien repartido. Todos hemos arrimado el hombro y nos hemos apoyado mutuamente cuando alguien se atascaba con el código o la documentación, gracias a una comunicación excelente. Nos quedamos con una sensación muy positiva: la certeza de haber trabajado codo con codo y haber sacado adelante este proyecto siendo un equipo de verdad.

14. CONCLUSIÓN.

Llegados a esta recta final del proyecto, y haciendo retrospectiva de hasta dónde hemos podido llegar, el balance general es sumamente positivo. Hemos logrado construir un sistema funcional y complejo, aunque nuestro diseño conceptual original abarcaba características aún más ambiciosas que, por el momento, han quedado como líneas de trabajo futuro.

Teníamos planificada la integración de placas solares para garantizar una verdadera autonomía energética del robot, así como la incorporación de sensores físicos específicos para la detección ambiental. En este sentido, uno de los retos que tuvimos que replantear fue la detección de fuego y humo mediante inteligencia artificial. Finalmente, esta funcionalidad no se pudo integrar, ya que nos encontramos con la barrera técnica de no saber cómo simular estos elementos de manera realista y precisa, tanto dentro del entorno virtual de Gazebo como en las pruebas físicas.

A pesar de estas limitaciones técnicas, desarrollar Fireeye ha sido un proyecto enormemente entretenido y enriquecedor. Hemos valorado y disfrutado mucho la libertad creativa que se nos ha dado en la asignatura para poder escoger, idear y dar forma a nuestro propio concepto desde cero.

Creemos firmemente que este es un proyecto que aún no está terminado, pero que posee un potencial inmenso. Si lográsemos plasmar y desarrollar todas las ideas que tenemos en mente, estamos convencidos de que Fireeye podría llegar a ser un gran producto con viabilidad real en el mercado. Sin embargo, somos conscientes de la realidad, es un proyecto académico y nos hemos topado con las lógicas limitaciones de tiempo, recursos materiales y el alcance de nuestros conocimientos técnicos actuales.

Aun así, esta experiencia nos ha servido para darnos cuenta de todo lo que abarca la robótica y el desarrollo de software, dejándonos con una gran motivación y encantados de seguir aprendiendo para, en un futuro, ser capaces de llevar ideas tan grandes como está a la realidad.

TABLA HISTORIAL DE VERSIONES.

Versión	Autor	Descripción	Fecha
V1 – Sprint 0	Greysy Burgos	Fase de concepción del proyecto	27/02/2026
V2 – Sprint 1	Greysy Burgos	Lanzamiento del mundo y primera navegación	30/03/2026
V3 – Sprint 2	Greysy Burgos	Navegación completa y funcionamiento de rosWeb	19/04/2026

V4 – Sprint 3	Greysy Burgos	Web casi terminada y funcional, funcionalidad de reconocimiento de botellas	13/05/2026
V5– Sprint 4	Greysy Burgos	Funcionamiento de la IA, web finalizada y navegación del robot real	02/06/2026